

CollabCanvas Realtime - Technical Architecture Specification

Section 1: Executive Summary & Project Purpose

CollabCanvas is a high-performance, real-time vector whiteboard platform enabling thousands of simultaneous users to sketch, annotate, diagram, and collaborate on shared infinite canvases with sub-20ms synchronization latency.

Section 1.1: Architectural Tenets

Unlike traditional CRUD applications, CollabCanvas is structured around continuous bidirectional streaming.

State synchronization utilizes operational transformation and conflict-free replicated data types (CRDTs).

Explicit Tech Stack Overview:

- Frontend Client: Flutter Web (Dart), Skia / CanvasKit GPU renderer
- Streaming Gateway: Go (Gorilla WebSocket engine)
- Coordination Cluster: Elixir (Phoenix Channels on the Erlang BEAM VM)
- Background Worker: Rust (Actix-web vector rasterizer)
- Metadata API: Ruby on Rails 7
- Persistence: Redis Cluster (live board state), PostgreSQL (tenants & accounts), AWS S3 (snapshots)

CollabCanvas Realtime - System Components & Topology

Section 2: Component Specifications

1. Canvas Client App (canvas-client)

Technology: Flutter Web compiled to WebAssembly with CanvasKit rendering.

Captures high-frequency pen strokes, vector bezier curves, gestures, and cursor positions.

Maintains an optimistic local vector scene graph and streams delta patches over WebSockets.

2. Edge Connection Gateway (edge-gateway)

Technology: Go with epoll-based Gorilla WebSocket server.

Manages persistent TLS WebSocket handshakes from clients.

Maintains heartbeat liveness and demultiplexes incoming binary stroke payloads.

Routes stroke frames to Room Coordination Cluster via internal low-latency TCP sockets.

Exposes endpoint: WSS /ws/canvas/join.

3. Room Coordination Cluster (room-coordinator)

Technology: Elixir / Phoenix Framework running on BEAM.

Utilizes lightweight OTP GenServers per active canvas room.

Coordinates room presence, peer broadcast, and merges vector conflict states.

Writes atomic delta operations to Redis Cluster and dispatches peer broadcast packets.

4. Vector Snapshot Worker (snapshot-worker)

Technology: Rust with headless SVG rasterizer and Actix-web.

Periodically drains room mutation streams, constructs lossless SVG / PNG vector thumbnails, and streams serialized board snapshots directly to Amazon S3.

5. Account & Permissions API (accounts-api)

Technology: Ruby on Rails 7 API mode.

Manages organization workspaces, billing licenses, user roles, and team invites.

Persists relational enterprise data to PostgreSQL.

Exposes endpoints: POST /api/v1/workspaces, GET /api/v1/workspaces/:id/members.

CollabCanvas Realtime - Data Persistence & Streaming Topology

Section 3: Storage & Memory Hierarchy

1. Live Board State Store (redis-state-store)

Type: In-Memory Key-Value & Pub/Sub (Redis 7 Cluster).

Purpose: Holds ephemeral vector scene graphs, active room cursors, and pub/sub channels for inter-node broadcasting.

2. Relational Metadata Database (postgres-accounts-db)

Type: Relational Database (PostgreSQL 15).

Purpose: Stores organization hierarchy, user credentials, team access control lists, and audit logs.

3. Object Storage Depot (s3-asset-storage)

Type: Cloud Object Store (Amazon Web Services S3).

Purpose: Long-term durable storage for frozen board snapshots, imported vector PDFs, and exportable high-res PNGs.

Section 3.1: Communication Matrix

- Client -> Edge Gateway: WSS /ws/canvas/join (Binary Protobuf over WebSocket)
- Edge Gateway -> Room Coordinator: Internal TCP Socket streaming
- Room Coordinator -> Redis State Store: Redis RESP3 commands & Pub/Sub
- Snapshot Worker -> S3 Asset Storage: AWS S3 REST API (HTTPS)
- Client -> Accounts API: HTTPS / REST for organization settings

CollabCanvas Realtime - Data Flow & Operational Analysis

Section 4: Real-Time Vector Synchronization Flow

Flow 1: Collaborative Vector Stroke Synchronization

Step 1: Canvas Client App captures stylus input, generates delta vector packet, and sends via WSS to Edge Gateway.

Step 2: Edge Gateway verifies room session token and passes binary frame to Room Coordination Cluster.

Step 3: Room Coordination Cluster OTP GenServer applies CRDT transform and writes delta to Redis State Store.

Step 4: Room Coordination Cluster broadcasts transformed delta stroke to all other connected room participants over WebSocket.

Step 5: Receiving Canvas Clients update their local Skia scene graphs with zero full-page reload.

Section 4.1: Documented Gaps & Unknowns

1. The exact CRDT mathematical algorithm (Yjs vs Automerge vs custom LWW-element-set) is not specified in this document.
2. WebRTC peer-to-peer mesh fallback is not detailed.
3. Voice / audio conferencing integration is not mentioned in documentation.